



Criteria API

*José Pedrajas de la Torre
Pedro Jesús Cros Pérez
Fco. Javier Martín-Lunas Escobar*



Índice de contenidos

1. ¿Qué es?
2. Esquema general
3. Introducción
4. Ventajas y desventajas
5. JPQL vs Criteria API
6. Usos de: *Where*, *OrderBy* y *Filter*
7. IAG'S
8. Bibliografía



¿Qué es?

Criteria API forma parte de la especificación JPA.

- Método seguro para crear consultas
- Recuperar entidades de una base de datos relacional
- Alternativa al uso de cadenas JPQL (Java Persistence Query Language)



Está a vuestro alcance

EntityManager.class

```
public interface EntityManager extends AutoCloseable { 13 usages
    void persist(Object var1);

    <T> T merge(T var1);

    void remove(Object var1);

    <T> T find(Class<T> var1, Object var2);

    <T> T find(Class<T> var1, Object var2, Map<String, Object> var3);

    <T> T find(Class<T> var1, Object var2, LockModeType var3);

    <T> T find(Class<T> var1, Object var2, LockModeType var3, Map<String, Object> var4);

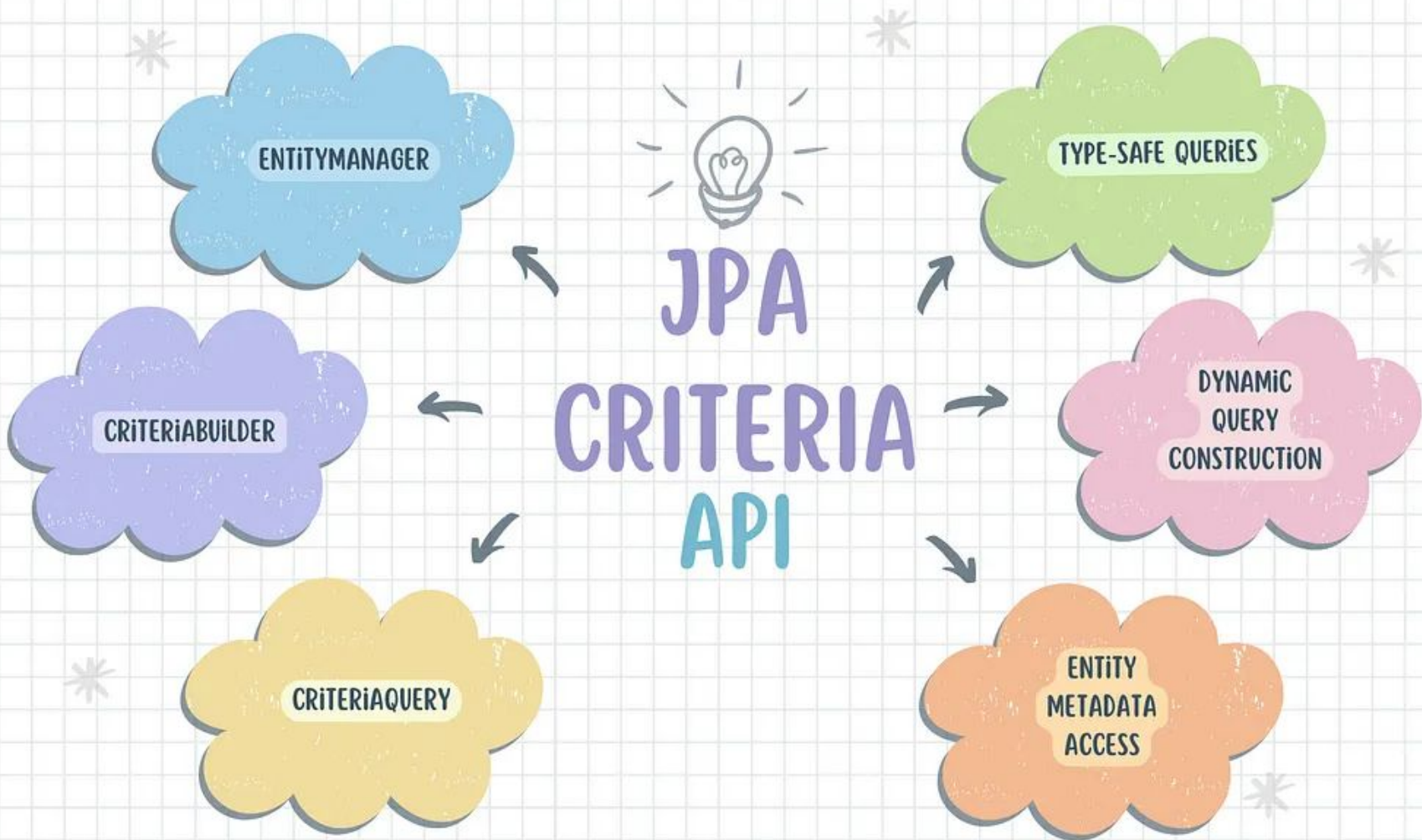
    <T> T getReference(Class<T> var1, Object var2);

    void flush();

    void setFlushMode(FlushModeType var1);
```

```
CriteriaBuilder getCriteriaBuilder();
```

```
Metamodel getMetamodel();
```





Componentes básicos

- ❖ EntityManager
- ❖ CriteriaBuilder
- ❖ CriteriaQuery
- ❖ Root
- ❖ Predicate
- ❖ Selection

● ● ● ReceiptDAOJpa.java

```
@PersistenceContext(unitName = "crmDB")  
private EntityManager em;
```

● ● ● ReceiptDAOJpa.java

```
public List<Receipt> searchByUserOrClient(Long idUserCrm, Long idClient) {  
    try {  
        CriteriaBuilder cb = em.getCriteriaBuilder();  
        CriteriaQuery<Receipt> cq = cb.createQuery(Receipt.class);  
        Root<Receipt> root = cq.from(Receipt.class);  
  
        Predicate p1 = null;  
        Predicate p2 = null;  
        if (idUserCrm != null) {  
            p1 = cb.equal(root.get("idUserCrm"), idUserCrm);  
        }  
        if (idClient != null) {  
            p2 = cb.equal(root.get("idClient"), idClient);  
        }  
  
        Predicate combined;  
        if (p1 != null && p2 != null) {  
            combined = cb.or(p1, p2);  
        } else if (p1 != null) {  
            combined = p1;  
        } else if (p2 != null) {  
            combined = p2;  
        } else {  
            combined = cb.conjunction();  
        }  
  
        cq.select(root).where(combined);  
        return em.createQuery(cq).getResultList();  
    }  
}
```

Filter

```
ReceiptDAO.jpjava
@OVERRIDE pjc0005
public List<Receipt> filter(
    Long      userId
    ,Long      clientId
    ,Double    minTotal
    ,Double    maxTotal
    ,LocalDate startDate
    ,LocalDate endDate
) {

    CriteriaBuilder cb = em.getCriteriaBuilder();
    CriteriaQuery<Receipt> cq = cb.createQuery(Receipt.class);
    Root<Receipt> root = cq.from(Receipt.class);

    List<Predicate> filters = new ArrayList<>();

    if (userId != null) {
        filters.add(cb.equal(root.get("idUserCrm"), userId));
    }
    if (clientId != null) {
        filters.add(cb.equal(root.get("idClient"), clientId));
    }
    if (minTotal != null) {
        filters.add(cb.ge(root.get("totalPrice"), minTotal));
    }
    if (maxTotal != null) {
        filters.add(cb.le(root.get("totalPrice"), maxTotal));
    }
    if (startDate != null) {
        filters.add(cb.greaterThanOrEqualTo(root.get("date"), startDate));
    }
    if (endDate != null) {
        filters.add(cb.lessThanOrEqualTo(root.get("date"), endDate));
    }

    cq.where(cb.and(filters.toArray(new Predicate[0])))
        .orderBy(cb.desc(root.get("date")));

    return em.createQuery(cq).getResultList();
}
```

Ejemplo de uso:

The screenshot shows a CRM application interface. At the top, there's a header with a logo and the text "CRM". Below the header, there's a filter form with the following fields:

- Usuario:** A dropdown menu with "Usuario" selected.
- Criterios total mínimo:** A text input field with "Ej. 0.00" and a unit "€".
- Criterios total máximo:** A text input field with "Ej. 1000.00" and a unit "€".
- Fecha desde:** A date input field with a calendar icon.
- Fecha hasta:** A date input field with a calendar icon.

Below the filter form, there's a button labeled "Filtrar". Below the button, there's a table with the following columns:

	Precio total	Fecha Alta	Empleado	Detalles del Cliente	Acciones
>	14377.0 €	2025-05-07	Daw daw2@red.ujae.es 23123454N	Luis Gómez luis.gomez@red.ujae.es	
>	20207.62 €	2025-05-07	Daw daw2@red.ujae.es 23123454N	Luis Gómez luis.gomez@red.ujae.es	
>	21314.84 €	2025-05-07		Maria Fernández maria.fernandez@red.ujae.es	

Red and blue arrows connect the Java code on the left to the corresponding UI elements on the right:

- Red arrow from `userId` to the **Usuario** dropdown.
- Red arrow from `clientId` to the **Usuario** dropdown.
- Blue arrow from `minTotal` to the **Criterios total mínimo** input field.
- Blue arrow from `maxTotal` to the **Criterios total máximo** input field.
- Blue arrow from `startDate` to the **Fecha desde** input field.
- Blue arrow from `endDate` to the **Fecha hasta** input field.

Entrando en profundidad

...

```
        ,Double maxTotal
        ,LocalDate startDate
        ,LocalDate endDate
    ) {

        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Receipt> cq = cb.createQuery(Receipt.class);
        Root<Receipt> root = cq.from(Receipt.class);

        List<Predicate> filters = new ArrayList<>();

        if (userId != null) {
            filters.add(cb.equal(root.get("idUserCrm"), userId));
        }

        if (clientId != null) {
            filters.add(cb.equal(root.get("idClient"), clientId));
        }

        if (minTotal != null) {
            filters.add(cb.ge(root.get("totalPrice"), minTotal));
        }

        if (maxTotal != null) {
            filters.add(cb.le(root.get("totalPrice"), maxTotal));
        }

        if (startDate != null) {
            filters.add(cb.greaterThanOrEqualTo(root.get("date"), startDate));
        }

        if (endDate != null) {
            filters.add(cb.lessThanOrEqualTo(root.get("date"), endDate));
        }

        cq.where(cb.and(filters.toArray(new Predicate[0])))
            .orderBy(cb.desc(root.get("date")));

        return em.createQuery(cq).getResultList();
    }
}
```

Entity Manager

```
Predicate.class
public interface Predicate extends Expression<Boolean> {
    BooleanOperator getOperator();

    boolean isNegated();

    List<Expression<Boolean>> getExpressions();

    Predicate not();

    public static enum BooleanOperator {
        AND,
        OR;

        private BooleanOperator() {
        }
    }
}
```

```
TypedQuery.class
public interface TypedQuery<X> extends Query { 9 usages
    List<X> getResultList();

    default Stream<X> getResultStream() { return this
    X getSingleResult();

    TypedQuery<X> setMaxResults(int var1);

    TypedQuery<X> setFirstResult(int var1);
}
```



Ventajas de usar Critería API

Mantenimiento más sencillo:

Al ser una API basada en objetos, las consultas se pueden refactorizar más fácilmente. Si cambian los nombres de las propiedades de las entidades, la Critería API se actualizará automáticamente.

Evita errores comunes de SQL:

Cómo se construye de manera programática, se eliminan problemas como inyecciones de SQL o errores de sintaxis, ya que no dependes de cadenas de texto para las consultas.

Facilita la reutilización:

El código se puede estructurar de forma modular, reutilizando componentes como filtros y condiciones sin duplicar código.

Soporte para consultas dinámicas:

Las condiciones de las consultas pueden cambiar en tiempo de ejecución sin necesidad de crear nuevas consultas en cada caso, lo que es útil para crear interfaces de usuario flexibles con múltiples filtros.



Desventajas de usar Critería API

Mayor complejidad:

A pesar de ser más segura y flexible, la Critería API puede ser más compleja de usar que una simple consulta JPQL o SQL. Puede resultar difícil al principio, especialmente para consultas muy sencillas.

Menor legibilidad:

Las consultas construidas mediante Critería API pueden resultar menos legibles que las consultas SQL tradicionales, especialmente en consultas complejas. Aunque es más seguro, el código puede ser más difícil para alguien sin experiencia en la API.

No es adecuada para todas las situaciones:

Para consultas simples, puede ser innecesario usar Critería API, ya que la solución tradicional de JPQL o SQL directo puede ser más eficiente y flexible.

Consulta usando JPQL

```
@Override 2 usages 8 jpt00012
public List<Payslip> getAllPayslips() {
    List<Payslip> payslips = null;
    try{
        TypedQuery<Payslip> query = em.createQuery(s: "SELECT p FROM Payslip p", Payslip.class);
        payslips = query.getResultList();
    }catch(Exception e){
        logger.log(Level.SEVERE, e.getMessage());
    }
    return payslips;
}
```

Consulta usando Criteria API

```
@Override 2 usages 2 jpt00012
public List<Payslip> getAllPayslips() {
    List<Payslip> payslips = null;
    try {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Payslip> cq = cb.createQuery(Payslip.class);
        Root<Payslip> root = cq.from(Payslip.class);
        cq.select(root);

        payslips = em.createQuery(cq).getResultList();
    } catch (Exception e) {
        logger.log(Level.SEVERE, e.getMessage(), e);
    }
    return payslips;
}
```

Método deleteUser usando Criteria API

```
@Override 3 usages ② Francisco Javier Martín-Lunas Escobar

public void deleteUser(Long id) {
    try {
        User user = em.find(User.class, id);
        if (user != null) {
            em.remove(user);
        }
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
}
```

```
@Override 3 usages ② Francisco Javier Martín-Lunas Escobar

public void deleteUser(Long id) {
    try {
        /* ----- 1. Eliminar las nóminas del usuario ----- */
        CriteriaBuilder cb = em.getCriteriaBuilder();

        CriteriaDelete<Payslip> deletePayslip = cb.createCriteriaDelete(Payslip.class);
        Root<Payslip> root = deletePayslip.from(Payslip.class);
        deletePayslip.where(cb.equal(root.get("userID"), id));

        // devuelve nº de filas afectadas; no lo necesitamos, pero ejecuta la orden
        em.createQuery(deletePayslip).executeUpdate();

        /* ----- 2. Eliminar el propio usuario ----- */
        User user = em.find(User.class, id);
        if (user != null) {
            em.remove(user);
        }
    } catch (Exception ex) {
        logger.log(Level.SEVERE, ex.getMessage(), ex);
    }
}
```

Uso del Where

```
public List<Payslip> getPayslipsByDateRange(Date startDate, Date endDate) { no usages new *
    List<Payslip> payslips = null;
    try {
        CriteriaBuilder cb = em.getCriteriaBuilder();
        CriteriaQuery<Payslip> cq = cb.createQuery(Payslip.class);
        Root<Payslip> root = cq.from(Payslip.class);

        cq.select(root).where(cb.between(root.get("date"), startDate, endDate));

        payslips = em.createQuery(cq).getResultList();
    } catch (Exception e) {
        logger.log(Level.SEVERE, e.getMessage(), e);
    }
    return payslips;
}
```



Uso del OrderBy

```
public List<Payslip> getPayslipsOrderedByDate(Boolean ascending) {  
    List<Payslip> payslips = null;  
    try {  
        CriteriaBuilder cb = em.getCriteriaBuilder();  
        CriteriaQuery<Payslip> cq = cb.createQuery(Payslip.class);  
        Root<Payslip> root = cq.from(Payslip.class);  
  
        if (ascending) {  
            cq.select(root).orderBy(cb.asc(root.get("date")));  
        } else {  
            cq.select(root).orderBy(cb.desc(root.get("date")));  
        }  
  
        payslips = em.createQuery(cq).getResultList();  
    } catch (Exception e) {  
        logger.log(Level.SEVERE, e.getMessage(), e);  
    }  
    return payslips;  
}
```

Uso de filtros

```
public List<User> buscarUsuariosAvanzada(String nombre, Integer edad, String correo, String ciudad) {
    CriteriaBuilder cb = em.getCriteriaBuilder();
    CriteriaQuery<User> cq = cb.createQuery(User.class);
    Root<User> root = cq.from(User.class);

    // Creamos una lista para almacenar las condiciones de filtro
    List<Predicate> filtros = new ArrayList<>();

    // Agregar filtros solo si los parámetros no son nulos
    if (nombre != null && !nombre.isEmpty()) {
        filtros.add(cb.like(root.get("nombre"), s: "%" + nombre + "%")); // Filtro por nombre
    }
    if (edad != null) {
        filtros.add(cb.equal(root.get("edad"), edad)); // Filtro por edad
    }
    if (correo != null && !correo.isEmpty()) {
        filtros.add(cb.like(root.get("correo"), s: "%" + correo + "%")); // Filtro por correo
    }
    if (ciudad != null && !ciudad.isEmpty()) {
        filtros.add(cb.like(root.get("ciudad"), s: "%" + ciudad + "%")); // Filtro por ciudad
    }

    // Si hay filtros, los añadimos a la consulta
    if (!filtros.isEmpty()) {
        cq.where(cb.and(filtros.toArray(new Predicate[0])));
    }
}
```

```
// Si hay filtros, los añadimos a la consulta
if (!filtros.isEmpty()) {
    cq.where(cb.and(filtros.toArray(new Predicate[0])));
}

// Crear la consulta y ejecutarla
TypedQuery<User> query = em.createQuery(cq);
return query.getResultList();
}
```



IAG's



ChatGPT $\Rightarrow$ o3



GitHub Copilot. Adaptar código.



Perplexity. Generación de código.



Bibliografía

- [Medium*](#) (Dana Prata)
- [Baeldung](#)
- [Arquitectura Java](#)
- [JakartaEE](#)
- [Tutorials Point](#)



Dana Prata

69 followers

Senior Software Engineer | Passionate About AI
and Clean Code